Computational Methods for Biological Modeling and Simulation (02512/02712)
Name: Jeffrey Wang
Andrew ID: jdw2
AI use statement:
Confirm my understanding of the problem.
Help translate my pseudocode into LaTeX.
Improve tricky phrasing when I want to describe how my algorithm works.
Debugging how to properly call code that expects a multi-lined input.

Problem Set 1

Problem 1 (15 points): The goal of this problem is for you to take informal descriptions of biological questions and cast them as well-known optimization problems covered in the reading. For each description, identify a classic computational problem we have studied that would solve the informal biological problem and describe how to convert inputs and outputs between the formal and informal problems.

To give an example, here is a similar problem from a prior year:

> Suppose we want to study differentiation of cell types during development of an organism. We examine expression of a large set of cell surface proteins and determine which ones are present in each cell type. Assume that each cell type expresses a different subset of the available cell surface proteins. We would then like to then create antibodies for a few of these proteins, attach fluorescent labels to these antibodies, and use them to label cells in an embryo. We would like to choose the antibody targets so that we can identify the type of each cell based on the labels attached to it. How can we find a minimum number of surface proteins sufficient to label every cell type?

A reasonable solution to this problem would be:

> We can cast this as a minimum test set problem. Our set S is the set of cell types and our collection of subsets C is the set of subsets of S corresponding to cells possessing each surface protein. The minimum test set will then identify a set of cell surface proteins that allows us to distinguish every cell type.

The systems to address are the following:

a. Suppose we are studying a metabolic network and are interested in how quickly a metabolite taken up by a cell could be converted to a downstream product by a series of enzymatic reactions. We assume there is a set of metabolites known to us and a particular source and destination metabolite. We further assume we know all of the allowed transformation reactions in the system and the maximum rate at which each can proceed. We want to know the minimum time at which a single molecule of the source metabolite could be converted into the destination metabolite.

We can cast this as a shortest path problem. Our set $v \in V$ is the set of metabolites where the source metabolite is s and the destination metabolite is t. Set $(u, v) \in E$ is the set of allowed transformation reactions with edge weights $w_{(u,v)}$ being the inverse of the maximum rate to reward reactions with

larger maximum rates to minimize total time. Then, the optimal sequence of reactions to go from metabolite $s \rightarrow t$ is the shortest path weight (found using Dijkstra's Algorithm) and summing across those edges will result in the minimum time.

b. Suppose we are studying a similar metabolic system to that above except that we might have many pathways by which the metabolite could be modified. Each reaction has some maximum throughput and we want to find the total throughput with which the destination metabolite can be produced if we have a sufficiently large quantity of the source metabolite.

We can cast this as a max flow problem. Our set $v \in V$ is the set of metabolites where the source metabolite is s and the destination metabolite is t. Set $(u, v) \in E$ is the set of allowed transformation reactions with max capacity $c_{(u,v)}$ being the maximum throughput. Then, following any edge with non-zero flow will reveal the interaction graph and summing incoming flow to t will result in the maximum total throughput.

c. Suppose we are trying to set up a bioreactor system, in which we rely on microorganisms to perform a series of reactions on a substrate to produce a product of interest. We set up a series of vats, each containing certain kinds of microorganisms, each of which we assume is capable of implementing one reaction of interest. Some microorganisms cannot co-exist, though, and we know certain pairs that cannot be in the same vat. We would like to set up as few processing vats as possible to allow the vats to contain all of the necessary microorganisms between them to produce the product. Assume the reactions can be performed in any order.

We can cast this as a graph coloring problem. Our set $v \in V$ is the set of microorganism species. Set $(u, v) \in E$ is the set of microorganism pairs that cannot co-exist. Then, graph coloring will assign all nodes $c \in C$ of k colors to ensure that no edge exists for all nodes in any color c. Finding the minimum k will result in the minimum number of processing vats needed.

d. Suppose we have are trying to dock a molecule into a binding pocket on a protein. There are a set of chemical groups on the molecule, each of which has some known binding energy for each binding pocket. We want to know the most stable (minimum energy) way of assigning chemical groups to binding pockets. Assume that each group can be bound to only one pocket and each pocket to only one group.

We can cast this as a bipartite matching with weights. Our set $a \in A$ is the set of chemical groups and set $b \in B$ is the set of binding pockets. Set $(a, b) \in E$ is the set of edges that connect $a \in A$ and $b \in B$ nodes with weight $w_{(a,b)}$ being the known binding energy. Then, bipartite matching will return us a subset $E' \subseteq E$ containing one-to-one assignment of each chemical group to each specific binding pocket. Minimizing for the sum of the edge weights $w_{(a,b)}, \forall (a, b) \in E'$ will result the optimized $E' \in E$, representing the minimum overall binding energy.

e. Suppose we are trying to design a vaccine and are interested in identifying proteins that produce an immune response through a certain antigen recognition protein (a protein that recognizes potentially foreign proteins). We identify a set of peptides that each trigger a response in the recognition protein. We believe that there is some pattern of amino acids, possible containing some variable-length gaps, that all of the recognized peptides must contain. Assume that every peptide recognized contains the same pattern, but possible different patterns of gaps within it. We would like to identify the most specific such pattern of amino acids.

We can cast this as a longest common subsequence problem. Our alphabet is the 20 existing amino acids and each peptide $p \in P$. Variable-lengthed gaps are already account for in LCS. Then, any common subsequence $c \in C$ represents a motif present in every peptide involved. Finding the c that maximized $|c|$ will result in the most specific such pattern of amino acids.

Problem 2 (20 points): In this problem, we will consider a problem in structural biology: assigning hydrogen bonds to a molecular structure. You can assume that we are given the three-dimensional structure of some protein. This protein will probably contain many atoms capable of forming hydrogen bonds, strong bonds that account for a large part of the energy holding a structure together. The atoms that form hydrogen bonds come in two flavors: donors (hydrogen atoms bound to nitrogen, oxygen, or fluorine) and acceptors (nitrogen, oxygen, or fluorine). A hydrogen bond can form between one donor and one acceptor, and each donor or acceptor can be involved in at most one bond. Whether a hydrogen bond will form between a given donor and acceptor will depend on how they are positioned relative to one another as well as whether either has access to a better binding partner. Assume the protein is flexible enough that binding of any one donor/acceptor pair will not affect the probability of binding of any other pair.

a. Assume we have identified every potential donor and acceptor atom in the structure and have a way to compute the energy of interaction between any given donor and acceptor should they form a hydrogen bond. Identify a classic optimization problem from the reading that could be used to model this problem and describe how to map our structure problem to the formal inputs and outputs of that optimization problem.

We can cast this as a bipartite matching. Let $a \in A$ be the set of every potential acceptor atom in the molecular structure and let $b \in B$ be set of every potential donor atom. Set $(a, b) \in E$ is the set of edges that connect acceptor a and donor b if they are capable of forming a hydrogen bond with weight $w_{(a,b)}$ being the computed energy of interaction. Then, bipartite matching will return us a subset $E' \subseteq E$ containing interactions that ensure that each donor or acceptor are in only 1 interaction. Minimizing the total energy of interaction over all valid pairings to produce the most stable set of hydrogen bonds and thus, the optimized subset $E' \in E$ will be the hydrogen bonds that form the native structure.

b. There is a complication that will potentially make our model from part a. invalid. Namely, atoms in the protein may hydrogen bond not just with one another but also with water molecules around the protein, which have both donor and acceptor atoms. Suppose we now assume that in addition to the information from part a., we also have an energy of hydrogen bonding to water for each potential donor and acceptor in the protein, which will vary from atom to atom depending on how exposed it is to the solution. Adjust your formalization of the problem in part a. to solve for this variant of the problem, while still allowing you to use the same classic algorithm.

We can still cast this as a bipartite matching in part a. How we might adapt the solution for part b is water molecules may act like both donors to acceptors and acceptors to donors in the existing system. Thus, let $c \in C$ be the set of all water molecules where $c_b \in C,\ b \in B$ and $c_a \in C,\ a \in A$ so that all existing donors and acceptors in the protein have a water molecule they could possibly interact with.
Now we add the following edges and weights:

1. **Protein-to-protein:** $w(a, b) =$ computed energy of interaction between proteins a and b

2. **Acceptor-protein-to-donor-water:** $w(c_a, a)$ = computed energy of interaction between protein a and water

3. **Donor-protein-to-acceptor-water:** $w(b, c_b)$ = computed energy of interaction between protein b and water

4. **Water-to-water:** $w(c_a, c_b) = 0$

This adjustment makes it so bipartite matching (minimizing weight) will still allow us to model the problem by allowing proteins to interact with each other given their calculated energy, proteins with water with their measured hydrogen bond, and water with water which would be neutral.

c. Another complication is that the answer may not be stable over time. It is possible that the protein may shift between many possible pairings of donors and acceptors over time. What we would often really want in a case like this is an *ensemble* describing all of the configurations of donors and acceptors that are close to the optimal energy. If our protein has a small number of donors and acceptors (say five of each), describe a practical algorithm that would handle this case.

Assuming that water is still involved in the problem, it is fair to say that due to the limited scope of the model, we can manually compute all possible configurations of the protein and identify the one that minimize the sum of energy. Let's say we have donor proteins $b \in B = \{b_1, \dots b_5\}$ and acceptor proteins $a \in A = \{a_1, \dots a_5\}$, then each donor b_i may either interact with one of the 5 acceptors a, water w, or remain free. We can iteratively match each donor with acceptors and the rest can either bond with water or remain free. Then calculating the energy $E(C)$ for some configuration $C = (u, v)$, we add up the energy of each bond (u, v) based on if they're both proteins or if either one is water, each with their established, pre-calculated interaction energy. The minimum is whatever C_{min} resulted in the smallest $E(C_{min})$ and using some threshold $e_{threshold}$, we can find all configurations C_i such that $E(C_i) \leq E(C_{min}) + e_{threshold}$.

I reason that this brute force method is do-able because the complexity is fairly small. Say for k protein-protein bonds that form in some configuration C_i, there are $\frac{5!}{k!(5-k)!}$ ways to select k donors out of the available 5 to form protein-protein bonds (this is n pick k). Among those k donors, we need to calculate how many possible acceptor pairs we can make. This is n permute k so $\frac{5!}{(5-k)!}$. Realizing that k spans $0-5$, this is a very manageable number of configurations needed to be computed.

d. Suppose we have a more complicated protein with perhaps a hundred donors and acceptors but we still want to know about the ensemble of near-optimal states. Determining the full ensemble may not be possible, however, since the number of near-optimal states could be very large. As a compromise, we decide that we want to find every hydrogen bond that is formed in any configuration that has energy within some value ΔG of the optimal. Suggest a way to find that using a polynomial number of applications of the algorithm from part b. (Hint: Think about how you would find the best possible energy given that a particular donor/acceptor pair is bound.)

Utilizing the hint, we can force the bond between donor protein b_i and acceptor protein a_j to then run the algorithm described in part b (removing those 2 proteins and their additional water nodes from the set). By this logic, we can identify the minimum energy configuration $C_{(b_i,a_j)}$ with that imposed bond (b_i, a_j) and its energy $E(C_{(b_i,a_j)}) = w(b_i, a_j) + E_{bm}(C_{B\setminus\{b_i\},\ A\setminus\{a_j\},\ W\setminus\{w_{b_i}, w_{a_j}\}})$ which is just the energy of that forced bond plus the bipartite matching resulting energy for the

rest of the configuration.

And then figuring out if at least one viable configuration with this forced bond has energy with ΔG of the optimal $E_{optimal}$ is as simple as $E(C_{(b_i,a_j)}) \leq E_{\text{opt}} + \Delta G$.

Estimations point to a complexity of running the b algorithm, which is the total number of proteins involved cubed, around the number of forced bonds we can test times. This is quite a lot of calculations that are needed to run but far from if you tried brute forcing this many possible involved donors and acceptors like we did in part c.

Problem 3 (20 points): In this problem, we will examine the tradeoffs involved in different ways of posing a common task in biological network analysis: finding groups of proteins likely to form a complex. We will assume that we have done an assay to assess which proteins interact with one another. Since such assays are generally very noisy, what we actually end up with is a probability of binding between each pair of proteins. We would then like to use these probabilities to identify modules of proteins mutually bound into larger complexes. We will try posing the problem in two different ways:

Model A. We first pose this problem by declaring that two proteins are part of a complex if there is chain of sufficiently probable pairwise binding interactions from one to the other. Assume we know the probability of any two genes interacting. For example, suppose A has probability 0.3 of interacting with B, B has probability 0.7 of interacting with C, and C has a probability 0.1 of interacting with A. Then if our cutoff is below 0.3 we would say A, B, and C are part of a complex. If the cutoff is between 0.3 and 0.7 then we would say B and C are part of a complex but A is not.

Model B. We next pose this problem with a stricter definition, saying that a set of proteins are in a complex only if every pair of genes in the module has a high probability of interacting. So, for example, proteins A, B, and C form a complex only if A is a has a high probability of interaction with B, B with C, and C with A.

a. Can we efficiently solve for model A to find all of the complexes modules? If so, describe in a few sentences an algorithm to solve for model A. If not, explain in a few sentences why we cannot find an efficient algorithm and describe a heuristic one could use to solve the problem in practice.

This is efficiently solvable because if we build a graph network with proteins as nodes and interactions that have a probability over the cutoff as edges, we can walk the graph in linear time to identify all connected proteins that form larger complexes.

b. Can we efficiently solve for model B to find all of the complexes? If so, describe in a few sentences an algorithm to solve for model B. If not, explain in a few sentences why we cannot find an efficient algorithm and describe a heuristic one could use to solve the problem in practice.

This is not efficiently solvable because if we build a graph network with proteins as nodes and interactions, finding these strict, highly interactive "clusters" of proteins is akin to checking for maximum cliques which is a known np-hard problem. Hierarchical clustering offers a heuristic way to form these clusters and classify them as such if we work bottom-up to greedily merge nodes together based on their interaction probabilities.

c. In judging which model is better, we need to consider the exact system and data source we are

using. Suppose we know that we need to examine a very large number of genes. Which model would that lead us to favor?

When imposed with the problem scope being a very large N, one might reason that opting for model B with an exponential time complexity is not worth it. Model A, as described before, has an attractive linear time complexity with methods like BFS/DFS to find connected components in the graph. Thus, that would lead us to favor model A.

d. Suppose we know that probability estimates are very inaccurate in a way that leads to a lot of false positives (incorrect estimates of high binding probability). Which model would that lead us to favor?

False-positives lead to noise in the graph dataset. I think model A is very susceptible in failing to account for these edges formed by incorrect estimates of high binding probability since all it takes is one strong connection to link proteins together. Model B would be more favorable as cliques demand that all nodes involved have connections with each other, making it less likely that one false-positive protein-protein connection could alter identified cliques unless that protein's connections with the rest are also false-positives.

e. Suppose we know that our data are inaccurate in a way that leads to many false negatives (saying two proteins have low probability of binding when they actually do bind). Which model would that lead us to favor?

False-negatives hurt both models significantly because, inversely, model B would fail to consider some protein p to be in a clique if we had a false-negative interaction with any other protein in it. Model A doesn't fail as hard as the traversal algorithm could still link p to the rest granted it has other true-positive edges with others. However, in the toy example described, a low threshold could completely sever A from the complex that B and C are apart of but if the probability interaction between A and C and/or A and B were false-negatives, then we missed out on including it into the complex. Nonetheless, we would favor model A.

Problem 4 (25 points): This is a programming problem in which we will examine a question related to biological network analysis. We will specifically consider a topic of concern in current network studies: the idea of *dynamic regulatory networks*, that may change over time. Suppose we are given a genetic regulatory network, in which we have identified transcription factors (TFs) that may regulate one another. We will assume, though, that rather than an interaction being present or not, it will have some probability of being active in a random condition. That is for any two TFs, we have a probability that each regulates the other. We can represent these by a matrix whose rows and columns are proteins, where each element (i, j) of the matrix is the probability TF i regulates TF j. We would like to know the most likely chain of interactions by which any one TF could activate any other in the network. We can assume that the probability of any chain of interactions is the product of probabilities of all interactions on the chain.

For example, suppose we believe TF i regulates TF j with probability 0.5, TF j regulates TF k with probability 0.3, and TF i regulates TF k with probability 0.1. Then we would conclude that the most likely way TF i could influence TF k is through the chain $i \rightarrow j \rightarrow k$, with probability $0.5 * 0.3 = 0.15$. On the other hand, if we believed TF i regulated TF k with probability 0.4, then we would conclude that the chain $i \rightarrow k$, with probability 0.4, would be the most likely way for i to regulate k.

Your goal will be to consider some possible ways to solve this problem, finding the most probable chain of regulatory interactions to explain the influence of one given TF on another given TF in the network. You can ignore the possibility of self-regulation.

a. Pose the problem of finding the most probable chain of interactions for a given pair of TFs as a classic computational problem familiar to us. (Hint: Convert each probability to its negative log.)

We can cast this as a shortest path problem. Construct a graph G with nodes V being the TFs of the genetic regulatory network and directed edges E when the probability of $i \to j > 0$. Each edge $(i, j) \in E$ has the weight $-\ln(P(i \to j))$ which works to our favor as higher probabilities result in shorter distances and the inverse is true for lower probabilities. Minimizing the sum of the negative log weights results in a path that maximizes the probability of such chain. Then, we are given one source TF node s and solve for the most probable chain of regulatory interactions to get to the other source TF node t.

b. Identify an algorithm we could use to solve this problem.

We can use Djikstra's Algorithm which is the most optimal solution to shortest path with non-negative weights.

c. As with the intraspecies phylogeny problem we saw in class, the classic algorithm is likely to have the problem of ambiguity. That is, there may be multiple equally good solutions to the problem. Suppose we decide to deal with that issue as we did with the intraspecies phylogeny problem: by saying that instead of finding an arbitrary best path we will instead find the union of all maximum-probability paths. Starting from the algorithm you identified in part b, write pseudocode for a modified version of the algorithm that finds the union of all most-probable paths.

We can adapt Djikstra's Algorithm by allowing TFs to have multiple possible parental TFs that call it. With the additional of many possible most-probable paths that share the same max total probability, for each edge (u, v), if it meets the same minimal distance as the parents of v, we can include u in its parental set. We will replace the parental set with u if it's shorter or change nothing otherwise.
Psuedocode next page.

d. Implement code for your algorithm of part c. You can assume the input to the problem has the following format:

```
<number of TFs> <starting TF> <ending TF>
<arbitrary number> <-log(p(0,1))> <-log(p(0,2))> <-log(p(0,3))> ...
<-log(p(1,0))> <arbitrary number> <-log(p(1,2))> <-log(p(1,3))> ...
<-log(p(2,0))> <-log(p(2,1))> <arbitrary number> <-log(p(2,3))> ...
...
```

where $p(i, j)$ is the probability TF i regulates TF j. That is, the input specifies a problem size, a source and a sink TF, and a matrix of minus log probabilities of direct infleunces between TFs. The code should output the set of edges contained in the union of most probable chains between the starting and ending TFs.

e. Design two test cases to demonstrate your code: one having a single best chain, and one having

Algorithm 1 Most-Probable Regulatory Chain

Require: Directed graph $G = (V, E)$ with edge weights $w(u, v) = -\ln P(u \to v)$, source TF $s \in V$, and target TF $t \in V$.

Ensure: E_{union}, the set of edges forming the union of all most-probable paths from s to t.

▷ **Modified Dijkstra with multi-parent tracking**

$U \leftarrow V \setminus \{s\}$
$\texttt{key}[s] \leftarrow 0$
$\texttt{parents}[s] \leftarrow \emptyset$
for each $u \in U$ **do**
  $\texttt{key}[u] \leftarrow \infty$
  $\texttt{parents}[u] \leftarrow \emptyset$
end for
for each $(s, u) \in E$ **do**
  $\texttt{key}[u] \leftarrow w(s, u)$
  $\texttt{parents}[u] \leftarrow \{s\}$
end for
while $U \neq \emptyset$ **do**
  Pick $u \in U$ that minimizes $\texttt{key}[u]$
  if $\texttt{key}[u] = \infty$ **then**
    break ▷ remaining nodes $\in U$ unreachable
  end if
  $U \leftarrow U \setminus \{u\}$
  for each $(u, v) \in E$ **do**
    if $v \in U$ **then**
      if $\texttt{key}[u] + w(u, v) < \texttt{key}[v]$ **then**
        $\texttt{key}[v] \leftarrow \texttt{key}[u] + w(u, v)$
        $\texttt{parents}[v] \leftarrow \{u\}$ ▷ shorter path found
      else if $\texttt{key}[u] + w(u, v) = \texttt{key}[v]$ **then**
        $\texttt{parents}[v] \leftarrow \texttt{parents}[v] \cup \{u\}$ ▷ equally short path found
      end if
    end if
  end for
end while

▷ **Reversed BFS on union of edges on optimal $s - t$ paths**

$E_{\text{union}} \leftarrow \emptyset$
if $\texttt{key}[t] = \infty$ **then**
  return E_{union} ▷ t is unreachable from s
end if
Init queue $Q \leftarrow [t]$
Init set $\texttt{Visited} \leftarrow \{t\}$
while $Q \neq \emptyset$ **do**
  Dequeue v from front of Q
  for each $u \in \texttt{parents}[v]$ **do**
    $E_{\text{union}} \leftarrow E_{\text{union}} \cup \{(u, v)\}$
    if $u \notin \texttt{Visited}$ **then**
      $\texttt{Visited} \leftarrow \texttt{Visited} \cup \{u\}$
      Push u onto Q
    end if
  end for
end while
return E_{union}

exactly two equally good best chains. Turn in the two test cases along with the output of your code on each.

Test Case 1: One best chain
python3 hw1.py <<'EOF'
3 0 2
0.0 1.0 3.0
100.0 0.0 1.5
100.0 100.0 0.0
EOF
Expected Output:
Edges in the union of most probable paths from TF 0 to TF 2:
(0 –> 1), (w(0,1) = 1.0)
(1 –> 2), (w(1,2) = 1.5)

Test Case 2: Two best chains
python3 hw1.py <<'EOF'
4 0 3
0.0 0.5 0.9 5.0
100.0 0.0 100.0 0.8
100.0 100.0 0.0 0.4
100.0 100.0 100.0 0.0
EOF
Expected Output:
Edges in the union of most probable paths from TF 0 to TF3:
(0 –> 1), (w(0,1) = 0.5)
(0 –> 2), (w(0,2) = 0.9)
(1 –> 3), (w(1,3) = 0.8)
(2 –> 3), (w(2,3) = 0.4)

***Problem 5 (20 points):** With a trend towards greater lab automation, more biology labs may have to start concerning themselves with how to optimally design large numbers of experiments being run automatically in parallel. We will consider here an example of how that might work. Suppose your lab has purchased an automated liquid-handling machine, a kind of robotic lab machine that can run large numbers of experiments in parallel. We will imagine that the machine can execute runs in "batches" in which it can handle a fixed maximum number of experiments per batch with a fixed amount of time per batch. You have lots of experiments planned and would like to run them in as few batches as possible. However, some groups of experiments cannot be run in the same batch. For example, suppose experiments A, B, C, and D each need to use one third of the machine's capacity. Then you can run any three of them in one batch but not all four simultaneously. Or, suppose experiments E and F need to be run at different temperatures and cannot both be in the same batch. We would like to figure out how many batches we need to run given a set of constraints of this form, each specifying some subset of experiments that cannot use the liquid handling machine simultaneously.

a. Formalize the problem statement and pose it as an optimization problem, including a formal statement of your inputs, outputs, and objective function.

Let $E = \{e_1, \ldots, e_n\}$ be the set of experiments and integer $C \in \mathbb{Z}^+$ be the capacity number of experiments per batch. Let $I = \{I_1, \ldots I_m\}$, $I_i \subseteq E$ be the set of experiment subsets that have issues preventing them from being put in the same batch. Your output should be a set $B = \{B_1, \ldots, B_k\}$, $|B_i| \leq C$, $B_i \subseteq E$, $S_j \nsubseteq B_i$ containing $k \in \mathbb{Z}^+$ batches that are subsets of experiments. The goal should be to minimize k but still respect the problem constraints. They should be no bigger than the capacity and be a (super)set of any the I sets with issues. The union of all batches should be E with no repeated experiments, that is: $\bigcup_{i=1}^{k} B_i = E$, $B_i \cap B_{i'} = \emptyset$

b. Rephrase your optimization problem as a decision problem.

Given E, the set of experiments, C, the batch capacity, I, the subset of experiments known to have conflicting issues, and some k number of batches, does there exist a set of k defined batches B such that no batch exceeds the capacity $|B_i| \leq C$, no batch involves known issues $S_j \nsubseteq B_i$, and all experiments are ran once $\bigcup_{i=1}^{k} B_i = E$, $B_i \cap B_{i'} = \emptyset$?

c. If you have posed the problem correctly, the decision problem should be NP-complete. Prove the completeness of the problem by showing that it is checkable in polynomial time and that a known NP-complete problem can be reduced to it.

We can cast this as a graph coloring problem. We must find an optimal k number of colors (batches) for the set of nodes E (experiments) given the set of edges I (pairs of experiments with issues). If we were given some possibly valid B, we can easily verify the solution in polynomial time by verifying each experiment is involved and one batch only, each batch is less than or equal to the capacity, and no experiments with issues are in the same batch, proving that the problem is in NP.

I also informally reduced the problem to graph coloring earlier but formally now: with a positive, integer bound k, we build graph G with experiments as the nodes and any issue pair as the edges. To map the problem exactly to graph coloring, we can ignore C or set it to $C = |V|$ so that a batch could hold every experiment but proving that the reduced problem is still NP-hard would accomplish an even more constrained problem. Running graph coloring works because a valid assignment for k colors would ensure all experiments are assigned a batch that aren't the same if they share an issue with each other. Because graph k-coloring is NP-hard, so is this problem by reduction.

Thus, it is NP-complete.

d. Now suppose two of your lab members come to you with algorithms for the problem. One of them offers you an approximation algorithm that runs in seconds and guarantees a solution within 1.5 of the optimal. The other offers you a branch-and-bound algorithm that requires exponential time in the worst case but can handle moderately large problems within a few hours. In a few sentences, discuss the circumstances in which you would prefer the approximation algorithm and the circumstances in which you would prefer the branch-and-bound algorithm. (Hint: Consider what else we would need to know about the experimental system to answer this question.)

* Recall that the starred problems are only for the 02-712/03-712 students.

I would choose the approximation algorithm in circumstances where $|E|$ and/or $|I|$ is very large. Outside of computational constraints, the practicality must be account for too. If each batch needs to be recalculated quickly in this feedback loop and/or they're cheap and low-risk, the approximation algorithm would be much preferred. However, in the complete opposites of the hypothetical

constraints mentioned, the branch-and-bound algorithm is preferred with smaller problem sets defined. The batches take much longer to complete so the algorithm can run offline with more time, the cost of batches is far higher with bigger risks of failure so an optimal assignment is necessary.